Pion qTMDWF Measurement

Physics Note – Mature Reference Implementation

May 5, 2026

Abstract

We describe the pion quasi-transverse-momentum-dependent distribution with Wilson Flow (qTMDWF) measurement implemented in the `pion_qTMDWF_pyquda.py` module of the PyQUDA measurement framework. This is the mature, validated reference implementation that has been used on ALCF Aurora (Intel PVC GPUs, oneAPI/dpnp backend). Active development continues in the parallel module `pion_qTMD_vibe_develop.py`, which contains additional features (explicit PDF path contracts, source gamma labels, optimized Wilson index reordering, backend-independent gating with `xp.asnumpy()`). The "WF" designation refers to the Wilson Flow used in the gauge-field preparation stage, not in the contraction code itself. The module computes connected pion two-point and three-point qTMD (CG coordinate-gauge) correlation functions with boosted Gaussian smearing.

Contents

1 Physics Overview

The qTMDWF measurement is a variant of the quasi-TMD measurement in which the gauge configurations are prepared with Wilson Flow smearing prior to the propagator computation and contraction. The Wilson Flow is a smoothing operation that evolves the gauge fields under a diffusion-like equation,

$$\partial_\tau V_\mu(x,\tau) = -g_0^2 \frac{\delta S_W[V]}{\delta V_\mu(x,\tau)} V_\mu(x,\tau), \qquad V_\mu(x,0) = U_\mu(x), \tag{1}$$

where S_W is the Wilson gauge action. The flow time τ is a tunable parameter that controls the smearing radius; larger flow times produce smoother gauge fields, suppressing ultraviolet fluctuations while preserving long-distance physics.

The qTMDWF contraction code itself does *not* implement Wilson Flow – this is done upstream in the gauge-field preparation pipeline. The notation "qTMDWF" in the module and output tags distinguishes measurements computed on flowed configurations from those computed on unflowed (or HYP-smeared) configurations.

The physical observable is the pion matrix element of a nonlocal quark bilinear operator separated by a spatial displacement b:

$$\mathcal{O}_\Gamma(b) = \bar{d}(x+b)\,\Gamma\,\mathcal{W}(x, x+b)\,u(x), \tag{2}$$

where $\mathcal{W}(x, x+b)$ is the Wilson line connecting x to $x+b$ and Γ is a Dirac matrix. In the coordinate-gauge (CG) approximation used in this implementation, the Wilson line is taken as unity (no explicit gauge links are inserted), and the displacement is implemented as a lattice shift of the propagator.

The leading-twist qTMD correlator is extracted from the three-point function

$$C_3^\Gamma(b, P^z; t_{\text{sep}}, \tau) = \langle \pi(P^z, t_{\text{sep}})\,\mathcal{O}_\Gamma(b;\tau)\,\pi^\dagger(P^z, 0)\rangle\,, \tag{3}$$

where $\pi(P^z, t_{\text{sep}})$ creates a pion with momentum P^z at Euclidean time t_{sep} and the operator is inserted at time τ.

2 Relationship to `pion_TMD`

The mature `pion_qTMDWF_pyquda.py` and the development `pion_qTMD_vibe_develop.py` share a common physical and algorithmic foundation. This section details both the commonalities and the divergences.

2.1 Shared Structure

Both modules implement the same core physical content:

- Connected pion two-point and three-point qTMD contractions with a single quark line and a single antiquark line.
- The antiquark propagator is constructed via γ_5-hermiticity:

$$S_{\text{anti}}(x, x_0) = \gamma_5\, S_q(x, x_0)^\dagger\, \gamma_5. \tag{4}$$

- The three-point function uses the fixed-sink sequential-source method: a sequential propagator is computed from a sink at time t_{sep} with the desired sink interpolator, and the operator insertion is contracted along the forward quark line.
- The CG qTMD operator displaces the forward propagator in the transverse ($\hat{x}$ or $\hat{y}$) and longitudinal ($\hat{z}$) directions by integer lattice units.
- Boosted Gaussian smearing with momentum injection is applied to both quark and antiquark lines at the sink.

2.2 Key Simplifications in the Mature Implementation

The mature `pion_qTMDWF_pyquda.py` is simpler than the development version in several respects:

- **No PDF path.** The mature module contains class definitions for `create_PDF_Wilsonline_index_list()` and `create_fw_prop_PDF_GI()`, but the `pion_TMDWF_measurement` class does *not* expose a dedicated `contract_PDF()` method. The PDF contractions in the development version are separate from the qTMD contractions and support both CG (coordinate-gauge) and GI (gauge-invariant) PDF paths.
- **Momentum scan via range.** Instead of accepting explicit momentum lists (`qlist`, `plist`), the mature implementation constructs the momentum list from a contiguous range: `plist = [[0, 0, pz, 0] for pz in range(pzmin, pzmax)]`. All spatial momenta are in the z-direction only.
- **Same boosted smearing conventions.** Both modules use `pos_boost` and `neg_boost` to describe the quark and antiquark momentum injections, respectively, and `width` for the Gaussian kernel width.

3 Two-Point Function

3.1 Continuum Definition

The connected pion two-point function with source at $x_0 = (0, \mathbf{0})$ and sink at $x = (t, \mathbf{x})$ is

$$C_2^{\Gamma}(p,t) = \sum_{\mathbf{x}} e^{-i\mathbf{p}\cdot(\mathbf{x}-\mathbf{x}_0)} \; \mathrm{Tr}_{\text{spin, color}} \left[S_{\text{anti}}(x, x_0)\, \Gamma_{\text{sink}}\, S_q(x, x_0)\, \Gamma_{\text{src}} \right], \tag{5}$$

where Γ_{sink} runs over the 16 bilinear gamma structures (see Section 5), Γ_{src} is determined by the source gamma mode, and S_{anti} is the antiquark propagator obtained via γ_5-hermiticity.

3.2 Source Gamma Modes

The mature implementation supports three source gamma modes, specified by the `src_mode` parameter:

1. `"fixed_g5"` **(default):** $\Gamma_{\text{src}} = \gamma_5$ for all sink gamma structures. This is the standard pseudoscalar pion interpolator.
2. `"same_as_sink"`**:** $\Gamma_{\text{src}} = \Gamma_{\text{sink}}$, i.e., the same gamma matrix used at the sink is also used at the source.
3. `"dagger_of_sink"`**:** $\Gamma_{\text{src}} = \gamma_5\, \Gamma_{\text{sink}}^{\dagger}\, \gamma_5$, the γ_5-hermitian conjugate of the sink gamma.

In contrast, the development version additionally supports an arbitrary specific gamma label (e.g., `"Z"`, `"X"`).

3.3 Contraction Algorithm

Algorithm 3.3 outlines the two-point contraction as implemented in `contract_2pt_pion()`.

1. Apply boosted smearing to the forward and backward propagators at the sink:

$$S_f \leftarrow \text{boosted\_smearing}(S_f,\, w = \text{width},\, \text{boost} = \text{pos\_boost}), \qquad S_b \leftarrow \text{boosted\_smearing}(S_b,\, w = \text{width},\, \text{bo} \tag{6}$$

2. Construct the antiquark backward line via γ_5-hermiticity:

$$\bar{S}_b = \gamma_5\, S_b^\dagger\, \gamma_5. \tag{7}$$

3. Contract with the sink gamma list to form the intermediate:

$$I_\Gamma(x)_{\alpha\beta} = \sum_\mu \bar{S}_b(x)_{\alpha\mu}\, (\Gamma_{\text{sink}})_{\mu\beta}. \tag{8}$$

4. Contract with the forward propagator and source gamma:

$$C_2^\Gamma(x) = I_\Gamma(x)_{\alpha\beta}\, S_f(x)_{\beta\alpha'}\, (\Gamma_{\text{src}})_{\alpha'\alpha}. \tag{9}$$

5. Project to momentum space via Fourier phase:

$$C_2^\Gamma(p,t) = \sum_{\mathbf{x}} e^{-i\mathbf{p}\cdot\mathbf{x}}\, C_2^\Gamma(x), \tag{10}$$

   implemented as a global lattice sum (`core.gatherLattice`) after pointwise multiplication with the momentum phases.

6. On rank 0, save to HDF5 via `save_proton_c2pt_hdf5`.

4 Three-Point qTMDWF Function

4.1 Fixed-Sink Sequential Source Method

The three-point function is computed using the fixed-sink sequential-source method. A sequential propagator is constructed by placing a source at the sink time slice t_{sep}:

$$\eta_{\text{seq}}(y; p_f, t_{\text{sep}}) = \delta_{t_y, t_{\text{sep}}}\, e^{-ip_f\cdot(y-x_0)}\, \Gamma_{\text{seq}}\, S_{\text{anti}}(y, x_0), \tag{11}$$

where $\Gamma_{\text{seq}} = \gamma_5 \Gamma_{\text{sink}}^\dagger \gamma_5$ ensures the correct Dirac structure for the sequential inversion. This source is then inverted:

$$D\, S_{\text{seq}} = \eta_{\text{seq}}. \tag{12}$$

The sequential propagator is converted to an antiquark-like backward line,

$$S_{\text{seq, anti}}(x, x_0; p_f, t_{\text{sep}}) = \gamma_5\, S_{\text{seq}}(x, x_0)^\dagger\, \gamma_5, \tag{13}$$

and enters the three-point contraction.

4.2 CG qTMD Operator

In the coordinate-gauge (CG) approximation, the Wilson line is replaced by the identity, and the operator insertion reduces to a spatial displacement of the forward propagator:

$$\mathcal{O}_b\, S_q(x, x_0) = S_q\left(x + b_T \hat{e}_\perp + b_z \hat{e}_z,\, x_0\right), \tag{14}$$

where $\hat{e}_\perp \in \{\hat{x}, \hat{y}\}$ is the transverse direction and $\hat{e}_z$ is the longitudinal direction. The displacement is implemented as consecutive lattice shifts via the PyQUDA `prop.shift(distance, direction)` method. To minimize interpolation error from large jumps, the Wilson line index list is ordered so that successive shifts differ by at most one lattice unit whenever possible.

4.3 Contraction

The CG qTMD three-point contraction is performed as:

$$C_3^\Gamma(q, b; p_f, t_{\text{sep}}) = \sum_{\mathbf{x}} e^{-i\mathbf{q}\cdot(\mathbf{x}-\mathbf{x}_0)}\ \mathrm{Tr}_{\text{spin, color}}\left[S_{\text{seq, anti}}(x, x_0; p_f, t_{\text{sep}})\, \Gamma_{\text{sink}}\, \mathcal{O}_b S_q(x, x_0)\, \Gamma_{\text{src}}\right]. \tag{15}$$

In the mature implementation, the contraction is performed inline in the application layer rather than through a dedicated class method. The algorithm is:

1. Copy the backward propagator as the starting shifted propagator.
2. For each Wilson line index $W = [b_T, b_z, \eta, \perp\text{-dir}]$:
   (a) Compute the incremental shift from the previous index:
   $$\Delta b_T = b_T^{\text{curr}} - b_T^{\text{prev}}, \qquad \Delta b_z = b_z^{\text{curr}} - b_z^{\text{prev}}. \tag{16}$$
   (b) Apply the transverse shift: `prop.shift`(Δb_T, $\perp$-dir).
   (c) Apply the longitudinal shift: `prop.shift`(Δb_z, z-dir).
   (d) Construct the source-side product $G_{\text{src}}\,\gamma_5$ and sink-side product $\gamma_5\,\Gamma_{\text{sink}}$.
   (e) Contract:
   $$C_3^\Gamma(q, b) = \sum_{\mathbf{x}} e^{-i\mathbf{q}\cdot\mathbf{x}}\ \mathrm{Tr}\left[G_{\text{src}}\gamma_5\, S^\dagger_{\text{shifted}}\, \gamma_5\, \Gamma_{\text{sink}}\, S_f\right]. \tag{17}$$
   (f) Gather globally and append to the result list.

4.4 The "WF" in qTMDWF

It is important to emphasize that the "WF" (Wilson Flow) designation refers to the gauge-field preparation, *not* to any flow operation inside the contraction module. The Wilson Flow is applied to the gauge configurations before the measurement workflow begins. The contraction code is identical whether the gauge links are flowed, HYP-smeared, or unsmeared. The qTMDWF tag in the HDF5 output path (via `get_qTMDWF_file_tag`) distinguishes the flowed gauge ensemble from other ensembles.

5 Gamma Matrix Conventions

5.1 16 Bilinear Structures

Both the mature and development modules scan 16 gamma matrix structures at the sink. The labels and their physical interpretations are:

Table 1: Gamma matrix labels and their QUDA/PyQUDA bitmask mappings.

Label	**PyQUDA `gamma()` index**	**Dirac structure**
5	15	γ_5
T	8	γ_4 (temporal)
T5	7	$\gamma_4\gamma_5$
X	1	γ_1
X5	14	$\gamma_1\gamma_5$
Y	2	γ_2
Y5	13	$\gamma_2\gamma_5$
Z	4	γ_3
Z5	11	$\gamma_3\gamma_5$
I	0	$\mathbf{1}$ (identity)
SXT	9	$\sigma_{14} = \frac{i}{2}[\gamma_1, \gamma_4]$
SXY	3	$\sigma_{12} = \frac{i}{2}[\gamma_1, \gamma_2]$
SXZ	5	$\sigma_{13} = \frac{i}{2}[\gamma_1, \gamma_3]$
SYT	10	$\sigma_{24} = \frac{i}{2}[\gamma_2, \gamma_4]$
SYZ	6	$\sigma_{23} = \frac{i}{2}[\gamma_2, \gamma_3]$
SZT	12	$\sigma_{34} = \frac{i}{2}[\gamma_3, \gamma_4]$

The QUDA bitmask indexing uses a specific encoding where each bit of the 4-bit gamma matrix index corresponds to a spacetime direction: bit $0 \to \gamma_1$, bit $1 \to \gamma_2$, bit $2 \to \gamma_3$, bit $3 \to \gamma_4$. In the code, the ordered list of bitmask indices is:

```
pyquda_gammas_order = [15, 8, 7, 1, 14, 2, 13, 4, 11, 0, 9, 3, 5, 10, 6, 12]
```

5.2 Gamma Stack Construction

The 16 gamma matrices are stacked into a single array of shape $(16, 4, 4)$. In the mature implementation, this stacking is done inline inside `contract_2pt_pion()` using a loop over `my_pyquda_gammas`. The development version abstracts this into the helper function `_gamma_stack()`, which additionally uses `_asarray_on_queue()` to ensure correct backend placement.

6 Boosted Smearing

Both the mature and development modules use the same boosted smearing implementation from `boosted_smearing_pyquda.py`. The boosted smearing is a gauge-covariant Gaussian smearing with momentum injection, defined in the continuum as:

$$\tilde{\psi}(x) = \int d^3y \, e^{-\frac{(\mathbf{x}-\mathbf{y})^2}{2w^2}} \, e^{i\mathbf{k}\cdot(\mathbf{x}-\mathbf{y})} \, \psi(y), \tag{18}$$

where w is the Gaussian width (parameter `width`) and $\mathbf{k}$ is the boost momentum (parameters `pos_boost` for the quark line, `neg_boost` for the antiquark line).

6.1 Implementation Details

The boosted smearing is implemented via distributed FFT:

1. Compute global grid coordinates for the current MPI rank.
2. Build the real-space kernel:

$$K(\mathbf{x}) = \exp\left(-\frac{x^2+y^2+z^2}{2w^2}\right) \exp\left(2\pi i \left(\frac{k_x}{L_x}x + \frac{k_y}{L_y}y + \frac{k_z}{L_z}z\right)\right). \tag{19}$$

3. Forward FFT the kernel and the fermion field to momentum space.
4. Multiply pointwise: $\tilde{\psi}(k) \leftarrow \tilde{\psi}(k) \cdot \tilde{K}(k)$.
5. Inverse FFT back to position space.
6. The boosted smearing wrapper applies this operation to all spin-color components of a `LatticePropagator` via `getFermion`/`setFermion` iteration.

The current implementation assumes identity gauge (no gauge rotation before/after FFT), which is valid for Coulomb-gauge-fixed configurations.

6.2 Usage Convention

- `pos_boost`: Momentum injected into the quark (forward) line. Convention: $[k_x, k_y, k_z]$ in units of $2\pi/L$.
- `neg_boost`: Momentum injected into the antiquark (backward) line.
- `width`: Gaussian kernel width w in lattice units.

In the ALCF Aurora application (`pyquda_qTMDWF_k0.py`), both boosts are set to $[0, 0, 0]$ (no momentum injection), and the width is $w = 7.0$, providing substantial spatial smearing.

7 Code Implementation

7.1 Module: `pion_qTMDWF_pyquda.py`

The mature reference implementation is located at:

`pyquda_measurement_utils/pion_qTMDWF_pyquda.py`

It defines the class `pion_TMDWF_measurement` and the following module-level structures:

- `my_gammas`: List of 16 gamma matrix label strings.
- `my_pyquda_gammas`: List of 16 `gamma.gamma()` objects in the canonical order.
- `pyquda_gammas_order`: List of 16 integer bitmask indices.
- `G5`: The γ_5 matrix object, `gamma.gamma(15)`.

7.2 Class: `pion_TMDWF_measurement`

7.2.1 Constructor

```
def __init__(self, parameters):
    self.eta = parameters["eta"]
    self.b_z = parameters["b_z"]
    self.b_T = parameters["b_T"]
    self.pzmin = parameters["pzmin"]
    self.pzmax = parameters["pzmax"]
    self.plist = [[0, 0, pz, 0] for pz in range(self.pzmin, self.pzmax)]
    self.width = parameters["width"]
    self.pos_boost = parameters["pos_boost"]
    self.neg_boost = parameters["neg_boost"]
```

Parameters:

- `eta`: List of source-sink separations (typically `[0]`).
- `b_z`: Maximum longitudinal displacement in lattice units.
- `b_T`: Maximum transverse displacement in lattice units.
- `pzmin`, `pzmax`: Momentum range bounds. The momentum list is constructed as $\{[0, 0, p_z, 0] \mid p_z = \text{pzmin}, \ldots, \text{pzmax} - 1\}$.
- `width`: Gaussian smearing width w.
- `pos_boost`: Quark momentum boost $[k_x, k_y, k_z]$.
- `neg_boost`: Antiquark momentum boost $[k_x, k_y, k_z]$.

7.2.2 `contract_2pt_pion()`

```
def contract_2pt_pion(self, latt_info, prop_f, prop_b,
                      phases, tag, src_mode="fixed_g5"):
```

Performs the connected pion two-point contraction with sink smearing. Arguments:

- `latt_info`: PyQUDA lattice information object.
- `prop_f`: Forward (quark) propagator.
- `prop_b`: Backward (antiquark) propagator.
- `phases`: Momentum phases from `MomentumPhase.getPhases()`.
- `tag`: Output file tag for HDF5.
- `src_mode`: One of `"fixed_g5"`, `"same_as_sink"`, `"dagger_of_sink"`.

7.2.3 `create_TMD_Wilsonline_index_list_CG()`

```
def create_TMD_Wilsonline_index_list_CG(self):
```

Generates the list of Wilson line displacement indices for the CG qTMD path. Each index is a 4-tuple $[b_T, b_z, \eta, \perp\text{-dir}]$ where:

- $b_T \in [0, \texttt{b\_T}]$: transverse displacement.
- $b_z \in [-\texttt{b\_z}, \texttt{b\_z}]$: longitudinal displacement.
- η: always 0 (source-sink separation index, single value).
- $\perp$ -dir $\in \{0, 1\}$: 0 for $\hat{x}$ direction, 1 for $\hat{y}$ direction.

Returns two lists: `index_list_trans0` ($\perp$-dir = 0) and `index_list_trans1` ($\perp$-dir = 1). Each list is reordered to minimize the difference between adjacent indices, reducing the magnitude of incremental shifts.

The reordering algorithm:

1. Sort indices by (b_T, b_z).
2. Scan linearly; when a jump larger than 1 in either b_T or b_z is detected, search ahead for a better-matching index and swap.

This is the same algorithm as in the development version, except that the development version has refactored it into a private method `_reorder_wilson_indices()` and the mature version duplicates the reordering logic inline in `create_TMD_Wilsonline_index_list_CG()` under the local function `reorder_indices()`.

7.2.4 `create_fw_prop_TMD_CG()`

```
def create_fw_prop_TMD_CG(self, prop_f, W_index,
                          WL_indices_previous):
```

Applies an incremental spatial shift to the forward propagator for the CG qTMD path. The shift is computed as:

$$S_f \leftarrow S_f.\texttt{shift}(\Delta b_T, \perp\text{-dir}).\texttt{shift}(\Delta b_z, z\text{-dir}), \tag{20}$$

where $\Delta b_T = b_T^{\text{curr}} - b_T^{\text{prev}}$ and $\Delta b_z = b_z^{\text{curr}} - b_z^{\text{prev}}$.

7.2.5 `create_PDF_Wilsonline_index_list()`

```
def create_PDF_Wilsonline_index_list(self):
```

Generates Wilson line indices for the pure longitudinal PDF path ($b_T = 0$). The index list is $[0, b_z, 0, 0]$ for $b_z \in [0, \texttt{b\_z}]$ and $[0, -b_z, 0, 0]$ for $b_z \in [1, \texttt{b\_z}]$. This method is defined but not called by any dedicated `contract_PDF()` method in the mature implementation; PDF contract support is present only in the development version.

7.2.6 `create_fw_prop_PDF_GI()`

```
def create_fw_prop_PDF_GI(self, gauge, prop_f_pyq, W_index,
                          WL_indices_previous):
```

Implements the gauge-invariant PDF shift using the gauge field links. For each spin-color component of the propagator:

- $\Delta b_z = 0$: no shift.
- $\Delta b_z = +1$: apply `gauge.pure_gauge.covDev(fermion, 2)` (forward z-direction).
- $\Delta b_z = -1$: apply `gauge.pure_gauge.covDev(fermion, 6)` (backward z-direction).
- Other Δb_z: raise `ValueError` (only unit-step shifts are supported to ensure a well-defined gauge-invariant path).

This method is defined but not used by the mature implementation's contract methods; it exists for potential standalone use.

7.3 Application Layer: `pyquda_qTMDWF_k0.py`

The ALCF Aurora application demonstrates the mature module's usage pattern:

```
from pyquda_measurement_utils.pion_qTMDWF_pyquda import (
    pion_TMDWF_measurement, pyquda_gammas_order, my_pyquda_gammas
)
Measurement = pion_TMDWF_measurement(parameters)
```

The three-point qTMDWF contraction is performed inline in the application (not via a class method), iterating over Wilson line indices and performing explicit `xp.einsum` contractions for each shift.

8 Key Differences from `pion_qTMD_vibe_develop.py`

Table 2 summarizes the structural and functional differences between the mature reference implementation and the active development version.

Table 2: Comparison of mature (`pion_qTMDWF_pyquda.py`) and development (`pion_qTMD_vibe_develop.py`) implementations.

Aspect	**Mature (qTMDWF)**	**Development (vibe)**
Backend gating	`.get()` for gather-to-CPU (older PyQUDA API)	`xp.asnumpy()` for uniform backend dispatch
Wilson index reordering	Duplicated inline as local function `reorder_indices()`	Refactored as class method `_reorder_wilson_indices()`
Momentum list	`plist` from `range(pzmin, pzmax)` in z only	Explicit `p_2pt`, `qext`, `qext_PDF` lists with x, y, z support
Source gamma	3 modes only: `fixed_g5`, `same_as_sink`, `dagger_of_sink`	4 modes: the 3 above plus any specific gamma label (e.g., `"Z"`)
Contraction methods	`contract_2pt_pion()` only; 3pt contracted inline in application	`contract_2pt_pion()`, `contract_qTMD_CG()`, `contract_PDF()` all as class methods
PDF path support	PDF index/forward-prop helpers defined but no `contract_PDF()`	Full CG_PDF and GI_PDF contracts via `contract_PDF(gauge_invariant=...)`
Code modularity	Gamma stacking, backward line, source gamma built inline in `contract_2pt_pion()`	Abstracted to `_gamma_stack()`, `_meson_backward_line()`, `_source_gamma_stack()`
Einsum optimization	No `optimize=True` flag	`optimize=True` in all `xp.einsum()` calls
Time cut	Not performed in module (done in application)	`contract_qTMD_CG()` returns full time extent
Array transpose	Application transposes `[W, p, g, t]` before writing	Module contracts naturally to `[W, gamma, p, t]`; application transposes

8.1 Backend API Evolution

A notable technical difference is the backend dispatch mechanism:

- **Mature:** Uses `arr.get()` to transfer GPU arrays to CPU. This is the older PyQUDA API that works with `dpnp` arrays (Intel oneAPI).
- **Development:** Uses `xp.asnumpy(arr)` where `xp` is the dynamically detected array backend (`numpy`, `cupy`, `dpnp`). This provides uniform backend independence.

8.2 Wilson Line Index Reordering

Both implementations use the same reordering algorithm, but the mature version defines the reordering as a nested local function inside `create_TMD_Wilsonline_index_list_CG()`, while the development version has extracted it into a reusable class method. The reordering minimizes the Chebyshev distance between consecutive Wilson line indices, which reduces the magnitude of propagator shifts and improves numerical precision in the CG approximation.

9 Momentum Convention

9.1 Momentum List Construction

In the mature implementation, momenta are restricted to the z-direction and are generated from a contiguous range:

```
self.plist = [[0, 0, pz, 0] for pz in range(self.pzmin, self.pzmax)]
```

The corresponding momentum phase vector used for projection is:

```
p_2pt_xyz = [[0, 0, -v] for v in range(parameters["pzmin"], parameters["
    pzmax"])]
```

The negative sign in the phase vector is the standard convention: the Fourier phase is $e^{-i\mathbf{p}\cdot\mathbf{x}}$, so the phase generator receives the negative of the desired momentum.

9.2 Four-Component Momentum Convention

All momentum vectors in the code are four-component lists $[p_x, p_y, p_z, p_E]$, where the energy component p_E is always 0 (Euclidean time is treated separately via the time extent of the correlator). The spatial components are integer multiples of $2\pi/L$:

$$p_i = \frac{2\pi n_i}{L_i}, \qquad n_i \in \mathbb{Z}. \tag{21}$$

In the mature qTMDWF implementation, $n_x = n_y = 0$ always, and n_z is scanned over the range $[\texttt{pzmin}, \texttt{pzmax})$.

10 HDF5 Output

10.1 Output Writer

The mature qTMDWF measurement uses the shared I/O utility `io_corr.py` for HDF5 output. Two writer functions are relevant:

- **Two-point:** `save_proton_c2pt_hdf5(corr, tag, gammalist, plist)`. Despite the "proton" name, this function is used for all hadron two-point correlators. It writes data organized as:

  `SS/<gamma>/PX0PY0PZ<pz>`

  The source time is extracted from the output tag via regex and the time axis is rolled so that $t = 0$ corresponds to the source position.

- **Three-point:** `save_qTMDWF_hdf5_noRoll(corr, tag, gammalist, plist, W_index_list)`. Writes data organized as:

  `SP/<gamma>/PX0PY0PZ<pz>/<b_X or b_Y>/eta<eta>/bT<bT>/bz<bz>`

  where `b_X` or `b_Y` is selected by the fourth component of each Wilson line index $[b_T, b_z, \eta, \perp$ -dir].

10.2 File Tag Convention

The output file tag is constructed by `get_qTMDWF_file_tag()` with the pattern:

`<data_dir>/qTMDWF/<lat_tag>.qTMDWF.<cfg>.<ama>.<src>.<sm>`

The `.qTMDWF` infix distinguishes these files from standard qTMD outputs (which use `.qTMD`), enabling downstream analysis to identify the Wilson Flow gauge preparation.

10.3 Data Shape Convention

The three-point qTMDWF data are saved with shape `[Wilson_index, momentum, gamma, time]`. The application performs a time roll (`np.roll(corr, -pos[3], axis=-1)`) so that $t = 0$ corresponds to the source position, ensuring consistent relative-time indexing across different source locations.

11 Application Workflow

11.1 Typical Measurement Pipeline

A complete qTMDWF measurement follows these steps (as exemplified in `pyquda_qTMDWF_k0.py`):

1. Initialize PyQUDA with MPI geometry and backend (dpnp/sycl for Intel GPUs).
2. Load the flowed gauge configuration and apply HYP smearing:

```
gauge = io.readNERSCGauge(gauge_path, ...)
gauge.hypSmear(1, 0.75, 0.6, 0.3, -1)
```

3. Create the Clover Dirac operator with multigrid solver parameters.
4. Generate source positions (either a single point or a distributed set).
5. For each source position:
   (a) Create a point source and apply boosted smearing.
   (b) Invert to obtain the forward propagator (smeared-point).
   (c) Compute the two-point function via `contract_2pt_pion()`.
   (d) Compute the three-point qTMDWF correlation functions by iterating over Wilson line indices and performing inline contractions.
   (e) Save results to HDF5, parallelized over gamma structures across MPI ranks.

11.2 MPI Parallelization Strategy

The code exploits two levels of MPI parallelism:

1. **Source-level:** Different source positions can be assigned to different MPI sub-partitions (not enforced in the code; typically sources are processed sequentially on all ranks).
2. **Gamma-level I/O:** The HDF5 writing is parallelized over gamma structures: each MPI rank writes a disjoint subset of the 16 gamma channels. This is implemented by cycling through `tasks[rank::size]` where `tasks` is $[0, 1, \ldots, 15]$.

All spatial contractions use `core.gatherLattice()`, which performs a distributed global sum across MPI ranks, reducing the 4D lattice field to a 1D time series.

12 Status and Usage Guidance

12.1 Mature Reference vs. Active Development

- `pion_qTMDWF_pyquda.py` (this document): The mature, validated reference implementation. It has been used in production on ALCF Aurora and is known to produce correct results. It uses the older PyQUDA API (`.get()`) and is less modular, but it is stable and well-tested.
- `pion_qTMD_vibe_develop.py`: The active development version. It adds backend-independent array dispatch (`xp.asnumpy()`), dedicated `contract_qTMD_CG()` and `contract_PDF()` methods, support for arbitrary source gamma labels, `optimize=True` einsum flags, full PDF path support (CG and GI), and better code modularity through helper functions.

12.2 Guidance for New Users

- For production runs on flowed gauge configurations: use the mature `pion_qTMDWF_pyquda.py` module with the `pyquda_qTMDWF_k0.py` application pattern.
- For development or when PDF path support is needed: use the `pion_qTMD_vibe_develop.py` module with the `Pyquda_pion_TMD_CG.py` application pattern.
- The boosted smearing, gamma conventions, and Wilson line index generation are identical between the two implementations.
- The HDF5 output format differs: qTMDWF uses `save_qTMDWF_hdf5_noRoll()` with `.qTMDWF` file infix, while the development version uses `save_qTMD_pion_hdf5_noRoll()` with `.qTMD` file infix.

A PyQUDA Propagator Layout

The PyQUDA propagator is stored in even-odd checkerboard layout with the following index order:

```
prop.data[cb, t, z, y, x_cb, spin_sink, spin_src, color_sink, color_src]
```

where `cb` is the checkerboard parity index (0 or 1), `t, z, y, x_cb` are the spacetime coordinates (with x split into even/odd sub-lattices), and the remaining four indices are the spin and color degrees of freedom at the sink and source.

B Global Lattice Sum

The `core.gatherLattice(array, axes)` function performs a distributed MPI sum over the specified spatial axes, reducing a distributed lattice field to a lower-dimensional tensor. For two-point functions, the call is:

```
core.gatherLattice(data, [2, -1, -1, -1])
```

where axis 2 (t) is kept and axes -1 (the three spatial dimensions) are summed over.

C Notation Glossary

Table 3: Common notation used in this document.

Symbol	**Meaning**
$S_q(x, y)$	Quark propagator from source y to sink x
$S_{\text{anti}}(x, y)$	Antiquark propagator: $\gamma_5 S_q(x, y)^\dagger \gamma_5$
Γ_{sink}	Dirac matrix at the pion sink
Γ_{src}	Dirac matrix at the pion source
$\mathcal{O}_b$	Nonlocal qTMD operator with displacement $b = (b_T, b_z)$
$\perp$-dir	Transverse direction: 0 for $\hat{x}$, 1 for $\hat{y}$
w	Gaussian smearing width (`width`)
$\mathbf{k}_{\text{pos}}$, $\mathbf{k}_{\text{neg}}$	Momentum boosts for quark/antiquark lines (`pos_boost`, `neg_boost`)
η	Source-sink separation index (always [0] in current implementation)
p_f	Fixed sink momentum
x_0	Source position
t_{sep}	Source-sink time separation